

Kiro CLI has rapidly evolved into a powerful AI-powered development assistant that transforms how developers interact with their terminal environments. Based on the latest features and capabilities documented in AWS DevOps blog posts, this comprehensive guide compiles essential tips and tricks with practical usage examples to maximize your productivity with Kiro CLI.

- [Model Selection and Configuration](#)
- [Image Support](#)
- [Conversation Management and Persistence](#)
- [Command Line Integration and Autocompletion](#)
- [Model Context Protocol \(MCP\) and Extensibility](#)

Model Selection and Configuration

Tip 1: Dynamic Model Switching During Active Sessions

Switch models mid-conversation using `/model` followed by arrow key selection:

```
1 Kiro> /model
2 ? Select a model >
3   } claude-4-sonnet
4   claude-3.7-sonnet (active)
5   claude-3.5-sonnet
```



This allows transitioning from Claude 3.5 Sonnet for routine tasks to Claude Sonnet 4 for complex architectural decisions within the same session.

Tip 2: Model-Specific Command Usage

Start a session with Claude Sonnet 4 using:

```
1 kiro-cli chat --model claude-4-sonnet
```



Ideal for sessions requiring advanced reasoning from the start, such as designing distributed systems.

Tip 3: Setting Default Model Preferences

Configure default model globally:

```
1 kiro-cli settings chat.defaultModel claude-4-sonnet
```



Ensures all new sessions default to Claude Sonnet 4 while maintaining override capability.

Tip 4: Understanding Model Capabilities

Leverage Claude Sonnet 4 for SWE-bench tasks:

```
1 kiro-cli chat --model claude-4-sonnet
2 Kiro> Optimize this Python code for Lambda cold starts...
```



Demonstrates its 72.7% agentic coding performance for serverless optimizations.

Image Support

Tip 5: Supported Image Formats and Limitations

Upload architecture diagrams in terminal:

```
1 Drag-and-drop architecture.png into terminal
2 Kiro> Generate Terraform for this design
```



Processes JPEG/PNG/WEBP/GIF up to 10 images per request.

Tip 6: Architecture Diagram to Infrastructure as Code

Convert visual designs to CloudFormation:

```
1 kiro-cli chat
2 Kiro> [drag infrastructure-diagram.png] Create SAM template
```



Generates deployable templates from visual inputs in [upload design-mockup.jpg].

```
Outputs JSX/CSS with accessibility features.

**Tip 7: Visual Documentation Analysis**
Explain ER diagrams:
```bash
kiro-cli chat
Kiro> [drop er-diagram.png] Generate SQL schema
```

Produces normalized database schemas from visual inputs.

## Conversation Management and Persistence

### Tip 8: Conversation Persistence with Resume Functionality

Maintain project-specific history:

```
1 cd ~/projects/inventory-service
2 kiro-cli chat --resume
```



Restores previous session context when returning to directory.

### Tip 9: Manual Conversation State Management

Archive important discussions:

```
1 Kiro> /save design-review-convo
2 Kiro> /load design-review-convo
```



Preserves critical decision-making threads.

### Tip 10: Directory-Based Context Switching

Automatic context separation:

```
1 cd ~/projectA && kiro-cli chat --resume # Loads ProjectA context
2 cd ~/projectB && kiro-cli chat --resume # Loads ProjectB context
```



Prevents cross-project contamination.

## Command Line Integration and Autocompletion

### Tip 11: Comprehensive CLI Autocompletion

Get AWS CLI suggestions:

```
1 aws s3 cp
2 Suggested: s3://bucket-name/path/
```



Supports 100+ tools including kubectl and terraform.

### Tip 12: Natural Language to Shell Translation

Convert English to executable code:

```
1 kiro-cli translate "rotate S3 logs daily with CloudWatch"
```



Outputs complete Lambda + EventBridge solution.

### Tip 13: Inline Terminal Suggestions

Accelerate Docker workflows:

```
1 docker compose
2 Suggested: up, down, build, config
```



Real-time syntax help as you type.

## Model Context Protocol MCP and Extensibility

[AWS MCP Server](#) is a suite of specialized MCP servers that help you get the most out of AWS.

In this section, we will use the following MCP server:

#### 1. Prerequisites

##### 1.1. Install uv

```
1 curl -Lsf https://astral.sh/uv/install.sh | sh
```



##### 1.2. Install Python using uv python install 3.10

```
1 uv python install 3.10
```



#### ► Quick setup ⚡

2. To configure Kiro CLI, edit the `~/.kiro/settings/mcp.json` file:

```
1 mkdir -p ~/.kiro
2
3 # Remove existing mcp.json file if it exists
4 rm -f ~/.kiro/settings/mcp.json
5
6 # Create mcp.json file with the specified content
7 cat > ~/.kiro/settings/mcp.json << 'EOF'
8 {
9 "mcpServers": {
10 "awslabs.core-mcp-server": {
11 "command": "uvx",
12 "args": [
13 "awslabs.core-mcp-server@latest"
14],
15 "env": {
16 "FASTMCP_LOG_LEVEL": "ERROR"
17 },
18 "autoApprove": [],
19 "disabled": false
20 }
21 }
22 }
23 EOF
24
25 echo "MCP configuration file created at ~/.kiro/settings/mcp.json"
```



3. Start Kiro CLI to check if the MCP server is properly configured. If MCP is configured correctly, you should see the message below:

```
1 kiro-cli chat
```

